



**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САМАРСКИЙ ГОСУДАРСТВЕННЫЙ АЭРОКОСМИЧЕСКИЙ
УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)» (СГАУ)**

ПРИНЦИПЫ РАЗРАБОТКИ УЧЕБНЫХ ПРОГРАММ

**Методические указания
к лабораторному практикуму
по дисциплине «Программная инженерия»**

Самара, 2012-2024

Составитель: доц., к. т. н. Зеленко Л.С.

УДК 004.4 (075)

Принципы разработки учебных программ: Методические указания к лабораторному практикуму по дисциплине «Программная инженерия»/ *Л. С. Зеленко*. – Самара: Изд-во Самар. гос. аэрокосм. ун-та, 2012. - 64 с.

Методические указания предназначены для студентов, обучающихся по направлению 010300.62 - «Фундаментальная информатика и информационные технологии», которые выполняют лабораторный практикум по дисциплине «Программная инженерия». Методические указания включают в себя рекомендации по основным этапам выполнения работ при разработке учебных программных систем, приводятся примеры оформления документации. В них учтены требования действующих государственных стандартов и нормативных материалов министерства образования и науки Российской Федерации, а также рекомендации, изложенные в международных стандартах по разработке программного обеспечения.

Указания выполнены на кафедре программных систем.

Печатаются по решению редакционно-издательского совета Самарского государственного аэрокосмического университета им. академика С. П. Королева.

Рецензент - канд. техн. наук, доцент Симонова Е.В.

СОДЕРЖАНИЕ

Технология быстрой разработки приложений RAD	4
Лабораторная работа №1 Разработка технического задания на программную систему	7
Лабораторная работа № 2 Описание и анализ предметной области	13
Лабораторная работа № 3 Постановка задачи	16
Лабораторная работа № 4 Разработка структуры системы.....	19
Лабораторная работа № 5 Разработка спецификации требований.....	22
Лабораторная работа № 6 Разработка прототипа интерфейса пользователя системы.....	27
Лабораторная работа № 7 Разработка Информационно-логического проекта системы.....	31
Лабораторная работа № 8 Разработка алгоритмов обработки данных	43
Оформление отчета	45
Список использованных источников	48
Приложение А Структура пояснительной записки	50
Приложение Б Пример оформления реферата	52

ТЕХНОЛОГИЯ БЫСТРОЙ РАЗРАБОТКИ ПРИЛОЖЕНИЙ RAD

Один из подходов к разработке программного обеспечения (ПО) в рамках спиральной модели жизненного цикла (ЖЦ) – получившая широкое распространение методология (технология) быстрой разработки приложений RAD (*Rapid Application Development* – быстрая разработка приложений) [1, 2]. Данная модель очень хорошо подходит к разработке учебных программ, т.к. включает в себя три составляющие:

- небольшую команду программистов (от 2 до 4 человек);
- короткий, но тщательно проработанный производственный график (от 2 до 4 мес.);
- повторяющийся цикл, при котором разработчики по мере того, как приложение начинает обретать форму, запрашивают и реализуют в продукте требования, полученные через взаимодействие с заказчиком.

Команда разработчиков должна представлять собой группу студентов, имеющих опыт в анализе, проектировании, кодировании и тестировании ПО, которые способны хорошо взаимодействовать как внутри самой команды, так и с пользователями и/или заказчиками. В качестве заказчика и пользователя системы выступает преподаватель.

ЖЦ ПО [1] по технологии RAD состоит из **четырёх фаз** (рисунок 1):

1. Анализа и планирования требований;
2. Проектирования;
3. Построения;
4. Внедрения.

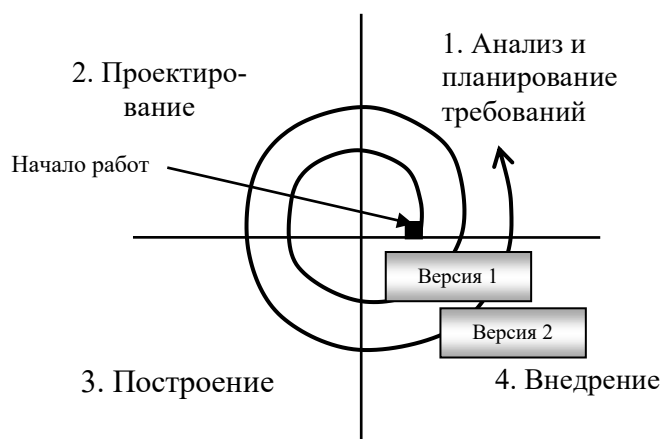


Рисунок 1 – Модель ЖЦ по технологии RAD

Рассмотрим адаптированный к учебному процессу вариант технологии RAD.

На **первой фазе анализа и планирования требований** преподаватель в вербальной форме формулирует постановку задачи: определяет функции, которые должна выполнять система, выделяет наиболее приоритетные из них, требующие проработки в первую очередь,

описывает информационные потребности (связи). На основании этих данных разработчики (под руководством преподавателя) формулируют требования к системе, которое фиксируется в виде технического задания (ТЗ) и подписывается всеми участниками проекта. В ТЗ ограничивается масштаб проекта, устанавливаются временные рамки для каждой из фаз. Для того, чтобы более четко и полно сформулировать постановку задачи, разработчики должны исследовать предметную область, в рамках которой выполняется разработка учебного проекта: познакомиться с основной терминологией, изучить информационные потоки объекта автоматизации, при необходимости изучить математический аппарат, который необходим для поддержки работы системы, познакомиться с основными достижениями в данной предметной области (найти и изучить основные функциональные возможности систем-аналогов).

Результатом фазы должны быть: список расставленных по приоритету функций будущей ПС; предварительная функциональная модель ПС; предварительная информационная модель ПС.

На **второй фазе проектирования** часть команды принимает участие в техническом проектировании системы под руководством преподавателя и, взаимодействуя с ним, уточняют и дополняют требования к системе, которые не были выявлены на предыдущей фазе. Более подробно рассматриваются *процессы системы*. При необходимости корректируется функциональная модель, создаются частичные прототипы: экранов, отчетов, устраняющие неясности или неоднозначности. Устанавливаются требования *разграничения доступа к данным*. На этой же фазе происходит определение необходимой документации. После детального определения состава процессов оценивается количество функциональных элементов разрабатываемой системы и ***принимается решение о разделении системы на подсистемы***. В подходе RAD каждый прототип развивается в часть будущей системы. Таким образом, на следующую фазу передается более полная и полезная информация.

Результатом данной фазы должны быть: общая информационная модель системы; функциональные модели системы в целом и подсистем; точно определенные интерфейсы между автономно разрабатываемыми подсистемами; построенные прототипы экранов, отчетов, диалогов и т.п.

На **третьей фазе построения** выполняется непосредственно сама быстрая разработка приложения (реализация подсистем). На данной фазе студенты производят итеративное построение реальной системы на основе полученных в предыдущей фазе моделей, а также требований нефункционального характера. Преподаватель на этой фазе оценивает получаемые результаты и вносит коррективы, если в процессе разработки система перестает

удовлетворять определенным ранее требованиям. Тестирование системы осуществляется в процессе разработки.

После окончания разработки подсистем производится постепенная интеграция каждой части системы с остальными, формируется полный программный код, выполняется тестирование системы в целом. Завершается физическое проектирование системы: определяется необходимость распределения данных; осуществляется анализ использования данных; производится физическое проектирование базы данных; определяются требования к аппаратным ресурсам; определяются способы увеличения производительности; завершается разработка документации проекта.

Результатом фазы является готовая система, удовлетворяющая всем согласованным требованиям.

На **четвертой фазе внедрения** производятся обучение пользователей, организационные изменения и параллельно с внедрением новой системы осуществляется работа с существующей системой (до полного внедрения новой). Учитывая, что технология RAD используется в рамках учебного процесса, данная фаза заменяется фазой «Доработка программной системы».

Технология RAD (как и любая другая) не может претендовать на универсальность, она хороша в первую очередь для относительно небольших проектов, разрабатываемых для конкретного заказчика, так называемое заказное ПО.

В заключение перечислим **основные принципы технологии RAD:**

- разработка приложений итерациями;
- необязательность полного завершения работ на каждом этапе ЖЦ;
- обязательное вовлечение пользователей на этапе разработки;
- тестирование и развитие проекта одновременно с разработкой;
- грамотное руководство разработкой, четкое планирование и контроль выполнения работ.

ЛАБОРАТОРНАЯ РАБОТА №1

РАЗРАБОТКА ТЕХНИЧЕСКОГО ЗАДАНИЯ НА ПРОГРАММНУЮ СИСТЕМУ

Во время лабораторного практикума должна быть разработана программная система (проект) средней сложности, при этом в соответствии с требованиями сегодняшнего дня разработка системы должна вестись не единолично, а командой разработчиков, каждый из которых в дальнейшем будет выполнять порученную ему часть проекта.

Тема проекта выдается преподавателем (в дальнейшем это руководитель проекта) на первом занятии, в соответствии с которой в течение первых двух недель команда студентов разрабатывает техническое задание по форме, приведенной в приложении А. Разработку ТЗ можно считать началом первой фазы ЖЦ будущей ПС.

Техническое задание разрабатывается в соответствии с ГОСТ 34.602-89 «Техническое задание на создание автоматизированной системы» [2] и в дальнейшем должно стать основным документом, по которому студенты ведут разработку проекта. Любые изменения ТЗ на систему должны быть согласованы с преподавателем и заверены его подписью.

Техническое задание состоит из трех частей:

- 1) содержание задания;
- 2) исходные данные;
- 3) календарный план выполнения работ.

Часть 1 – «Содержание задания». Данная часть ТЗ фиксирована, в ней перечислены основные составные этапы выполнения проекта. При разработке ПС в рамках лабораторного практикума и в дальнейшем при выполнении курсового проекта будут использоваться две основные методологии: объектно-ориентированного анализа и проектирования (ООАП) и методология структурного проектирования [3].

ООАП (Object-Oriented Analysis/Design) - технология разработки программных систем, в основу которых положена объектно-ориентированная методология представления предметной области в виде объектов, являющихся экземплярами соответствующих классов [4]. Методология ООАП тесно связана с концепцией автоматизированной разработки программного обеспечения (Computer Aided Software Engineering, CASE) и языком моделирования UML (Unified Modeling Language).

Методология объектно-ориентированного анализа и проектирования используется при выполнении лабораторной работы №2 – описании и анализе предметной области, где студенты должны выявить взаимосвязи между основными информационными объектами ПС, определить их характеристики и порядок взаимодействия.

Методология структурного проектирования применяется на первой фазе проектирования (лабораторная работа №3) при разделении системы на подсистемы, здесь используется принцип проектирования «сверху вниз». За каждым студентом закрепляется определенный перечень работ (обычно это разработка отдельной подсистемы), выполнение которых в дальнейшем контролирует преподаватель.

Часть 2 – «Исходные данные к проекту» включает в себя следующие подразделы:

1. Характеристики объекта автоматизации (или управления);
2. Требования к информационному обеспечению.
3. Требования к техническому обеспечению.
4. Требования к программному обеспечению.
5. Общие требования к проектируемой системе.
6. Перечень дополнительных работ (если необходимо).

Характеристики объекта автоматизации. Здесь указываются общие характеристики объекта автоматизации, характерные для рассматриваемой предметной области:

- a) полное название объекта (ов);
- b) условия его функционирования;
- c) *количественные и качественные показатели* объекта, которые являются ограничениями процесса функционирования.

В качестве примера рассмотрим проект «Автоматизированная система составления и разгадывания линейного кроссворда по выбранной теме», ТЗ на который приведено в приложении Б.

Понятие «объект автоматизации» в явном виде в ГОСТ 34.602-89 [2] нигде не определено, но если внимательно прочитать п. 2.4.1. «В подразделе «Назначение системы» указывают вид автоматизируемой деятельности (управление, проектирование и т. п.) и перечень объектов автоматизации (объектов), на которых предполагается ее использовать»¹, то из него следует, что под «объектами автоматизации» авторы понимали вовсе не процессы. Будем считать, что объектом автоматизации может быть только материальный (интеллектуальный) объект - организация, магазин, цех, отдел, и так далее.

Комментарии к примеру. Из названия темы следует, что в данном случае объект автоматизации – это линейный кроссворд, а виды автоматизируемой деятельности – это *процессы*:

- 1) составления/ генерирования кроссворда;
- 2) разгадывания кроссворда;

¹ ГОСТ 34.602-89, с. 3.

3) работы со словарем понятий.

Обращаю ваше внимание на то, чем составление кроссворда отличается от генерирования: в первом случае пользователь вручную составляет кроссворд (добавляет понятия, удаляет понятия и т.п.), во втором (генерирование) - кроссворд составляется системой автоматически в соответствии с теми настройками, которые выполнил пользователь. Процесс составления во многом дублирует функции, которые будет выполнять пользователь при редактировании кроссворда, поэтому процесс редактирования кроссворда не нужно выделять отдельно.

Для каждого процесса в ТЗ должны быть указаны количественные показатели-ограничения, для того, чтобы их правильно выбрать, необходимо знать начальные сведения о структуре самого объекта, каким образом будет проходить его построение и разгадывание. Отправной точкой является определение линейного кроссворда (ЛК). Итак, ЛК – это «цепочка слов, которая строится *методом стыкования*, где последняя буква первого слова является первой буквой второго и т. д. В чайнворде, как и в кроссворде, используются только имена существительные в именительном падеже и единственном числе» [5]. Так как ЛК строится из слов, то необходимо указать ограничение на длину слова: минимальную длину слова можно определить равную 3, а максимальную – 15, потому что чаще всего кроссворды будут составляться на бытовые темы и сам ЛК не должен быть очень простым. Чтобы ЛК было интересно разгадывать, он не должен быть очень коротким, поэтому минимальное количество слов, например, 5, а максимальное – 15. Идем дальше: «слова в чайнворде не пересекаются, а только *стыкуются* друг с другом. Иногда цепочку слов изгибают для придания сетке причудливой формы. Длинная изогнутая цепочка может неоднократно пересекать саму себя, как слова в кроссворде, такая головоломка обычно называется кроссчайнвордом». Исходя из этого, можно задать следующее ограничение – на форму отображения ЛК: обычная (линейная), спираль, змейка, W-образная. «В линейных кроссвордах слова могут перекрываться не только одной, но и двумя или тремя буквами, поэтому их длина указывается в скобках при определении к слову» [5], эта часть описания ЛК дает еще одно ограничение: количество букв в пересечении - от 1 до 3. В качестве пожелания, заказчик отметил, что ЛК необходимо строить в двух режимах: ручном и автоматическом (генерация кроссворда), из этого следует еще одно ограничение – составление кроссворда осуществляется с привязкой к словарю понятий. Для того чтобы системы была более универсальной (необходимо обеспечить создание тематических кроссвордов или на общие области знаний), заказчик предложил загружать в систему внешние словари понятий и обеспечить ему возможность редактировать их содержимое.

При разгадывании кроссворда могут возникнуть затруднения, поэтому (в соответствии с пожеланиями заказчика) в системе должна быть организована система подсказок, количество которых можно связать с количеством слов – не менее 1 и не более 10% от количества слов.

Требования к информационному обеспечению. Разработка информационного обеспечения (ИО) – наиболее важная часть проекта, она может оказать существенное влияние на весь процесс разработки, поэтому уже на стадии разработки ТЗ необходимо определить:

- 1) на основании каких документов разрабатывается методическое и информационное обеспечение системы (нормативные и другие документы);
- 2) перечень исходных данных:
 - a) какие массивы данных используются и в каких форматах;
 - b) на каких носителях эти данные будут поставляться в систему;
- 3) перечень выходных данных:
 - a) какие массивы данных будут являться результатом работы ПС;
 - b) какие документы будут представлены пользователю и в каком виде (указывается вид носителя) и с какой периодичностью;
 - c) какие требования по целостности данных и их защите должны быть выполнены в проектируемой системе.

Особо должны быть выделены файл-серверные и клиент-серверные части информационного обеспечения, если таковые имеются.

Комментарии к примеру. Для разработки данной системы никаких нормативных документов не требуется (стандартов, инструкций и т.п.), поэтому необходимо только сослаться на информацию, где определена структура и свойства ЛК, а также требования к его построению. Также необходимо определить требования по входным и выходным данным. Большинство параметров ЛК будут задаваться пользователем в режиме диалога: он обязательно должен подключить словарь понятий, из которого будут формироваться задания для кроссворда. В данном случае «словарь понятий» – это текстовый файл определенной структуры (каждая строка файла – это понятие и его расшифровка), который должен загружаться в систему из внешней памяти (с любого логического диска), с которым пользователь должен иметь возможность работать дополнительно. Обязательное условие заказчика – возможность создания коллекции ЛК, поэтому в системе должна быть предусмотрена возможность сохранения наиболее интересных кроссвордов в файл. На данном этапе еще трудно определить структуру этого файла (она должна учитывать и

возможность дальнейшего разгадывания кроссворда с помощью данной системы), поэтому можно написать, что «структура файла определяется в процессе проектирования». Обязательным условием составления любого типа кроссвордов является его целостность (для данного случая - это отсутствие пустых клеток в середине ЛК), поэтому это обязательно должно быть записано в требованиях.

Требования к техническому обеспечению. Здесь формулируются ограничения по составу технических средств автоматизации с указанием конкретных типов оборудования и ЭВМ или их составляющих, используемых в проекте, если они заранее известны. Иначе в этом разделе указывается, что состав комплекса технических средств системы определяется в процессе проектирования системы.

Требования к программному обеспечению. Здесь приводится перечень используемых системных и прикладных программных средств, включая операционную систему, систему программирования, систему управления базами данных (в случае необходимости) и другие инструментальные средства (например, среда проектирования) с точным наименованием версий, если они заранее известны. Иначе указывается, что состав программного обеспечения определяется в процессе проектирования системы. Дополнительно могут быть указаны требования по совместимости разрабатываемого программного обеспечения с существующими системами.

Общие требования к проектируемой системе. В данной части ТЗ отдельно выделяется подраздел «Функции, реализуемые системой». В нем приводится подробный перечень функций, которые должна выполнять проектируемая система (или подсистема) в процессе ее эксплуатации. Отдельно должны быть выделены функции ввода данных, их обработки, передачи, хранения, а также формирования отчетов с выдачей на экран или печатающие устройства, функции управления, работа со справочниками и различные сервисные (обслуживающие систему) функции. Формулировка функций должна быть однозначной и конкретной, так как именно она является основой приемки проекта руководителем и проверки на полноту и качество реализованной системы или подсистемы.

Комментарии к примеру. Функции, которые должна выполнять данная система, определяются в первую очередь видами автоматизируемой деятельности, которые были определены в п.2.1 ТЗ, часть из них уже была выявлена в ходе обсуждения ограничений на ЛК. Среди неявных функций, про которые не должны забывать разработчики, это:

- а) Визуализация процессов работы с кроссвордом;
- б) Выдача сведений о системе (справочные данные о системе и о том, как с ней работать).

Нужно отметить, что многие перечисленные в ТЗ функции, не раскрываются подробно, их необходимо будут детализировать при разработке функциональной спецификации (лабораторная работа №5).

В других подразделах оговариваются специальные технические требования, предъявляемые к системе:

- по быстродействию (времени реакции на выполнение наиболее важных функций);
- по режиму работы (диалоговый/интерактивный, автоматический);
- по точности (в случае, если в системе производятся математические расчеты, требующие минимизации вычислительных погрешностей, или используются внешние информационные источники (датчики, измерители и т.п.));
- по достоверности;
- по условиям функционирования (диапазон температур, относительная влажность, давление, наличие в атмосфере пыли, вредных примесей и т.д.),

а также все другие количественные и качественные показатели, определяющие эффективность функционирования системы. Кроме того, в данном разделе указываются санитарные правила и нормы (СанПин 2.2.2./2.4.2198-07 [6]) и ГОСТы, требования которых необходимо учитывать при разработке такого класса систем, с учетом того, что системы разворачиваются на средствах вычислительной техники [7, 8].

Часть 3 – Календарный план выполнения работ. Технология RAD, как уже говорилось выше, требует жесткого следования плану-графику работ, поэтому в ТЗ оговариваются ключевые задания, по которым преподаватель должен проводить обязательный контроль. Каждый из перечисленных этапов должен завершаться *полностью готовой документацией*, согласованной с заказчиком (руководителем). Невыполнение в срок какого-либо из этапов может привести либо к сдвигу «контрольных точек» по оставшимся этапам, либо к незавершению проекта в срок.

В заключение хотелось бы отметить, что процесс составления ТЗ на систему:

- 1) требует от разработчиков коллективных обсуждений и принятия ответственных решений;
- 2) позволяет выявить наиболее «узкие» места проекта и оценить возможные риски;
- 3) дает возможность команде разработчиков распределить между собой все виды выполняемых работ, сосредоточив в дальнейшем усилия на концептуальных аспектах проекта;
- 4) определить наиболее приоритетные функции, которые будут составлять каркас системы.

ЛАБОРАТОРНАЯ РАБОТА № 2

ОПИСАНИЕ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Как уже говорилось ранее, технология RAD включает в себя элементы методологии *объектно-ориентированного проектирования и анализа* предметной области. Для быстрой и эффективной разработки программной системы с минимальным браком требуется определить верное направление работы [9]. Для того чтобы правильно построить систему, сначала необходимо построить ее модель (этим вы будете заниматься позднее). Моделирование - это устоявшаяся и повсеместно принятая *инженерная методика*. Хорошая модель всегда включает элементы, существенно влияющие на результат, и не включает те, которые малозначимы на данном уровне абстракции. Модели строятся для того, чтобы лучше понимать разрабатываемую систему. При этом нужно помнить об основных принципах моделирования, один из которых гласит: «*Лучшие модели - те, что ближе к реальности*» [9]. Поэтому первое, с чего нужно начать разработку системы – это досконально изучить предметную область, в которой будете работать.

В соответствии с методологией ООАП выделяются следующие шаги работы над проектом (системой).

1. *Описание предметной области*. Определение гласит: «**Под предметной областью (application domain) принято понимать ту часть реального мира, которая имеет существенное значение или непосредственное отношение к процессу функционирования программы. Другими словами, предметная область включает в себя только те объекты и взаимосвязи между ними, которые необходимы для описания требований и условий решения некоторой задачи**» [4]. Выделение исходных и базовых компонентов предметной области, необходимых для решения той или иной задачи, представляет собой, в общем случае, нетривиальную задачу. Сложность проявляется в неформальном характере процедур или правил, которые нужно применять для этой цели. Для выделения или идентификации компонентов предметной области было предложено несколько способов и правил. Сам процесс получил название *концептуализации предметной области* [4].

Следовательно, разработчикам необходимо выделить основные объекты (компоненты), участвующие в функционировании системы, определить их наиболее существенные характеристики, взаимосвязи в рамках решаемой задачи, а также определить основные информационные потоки в системе. При этом отдельные компоненты выбираются таким образом, чтобы при последующей разработке их было удобно представить в форме классов и объектов. В этом случае немаловажное значение приобретает и сам язык представления информации о концептуальной схеме предметной области.

Сложность предметной области определяет количество объектов и связей между ними, поэтому описание должно включать в себя базовые *термины и определения*, сопровождаться различными примерами, в нем могут приводиться различного рода *классификации*, поясняющие различные свойства описываемых объектов. Если в системе используются математические модели, то они также должны быть описаны с учетом специфики применения.

Комментарии к примеру. Для разрабатываемой системы в первую очередь необходимо дать определение кроссворда [10], привести краткую историю его создания, рассказать о разновидностях кроссвордов и особенностях их построения и разгадывания (привести в качестве иллюстраций «сетки» различных кроссвордов)², при этом подробно рассказать об особенностях линейного кроссворда, различных формах его построения (привести иллюстрации). Это особенно важно, т.к. в техническом задании было зафиксировано 4 различные формы представления ЛК (на рисунках 1-2 представлены два варианта представления линейного кроссворда).

Сын Гермеса.						Колючая рыба.			Единица зем- ельной пло- щади.	
Приставка, означает - САМО				Сладкий спиртной напиток				Голубой		

Рисунок 1 – Чайнворд, как разновидность линейного кроссворда

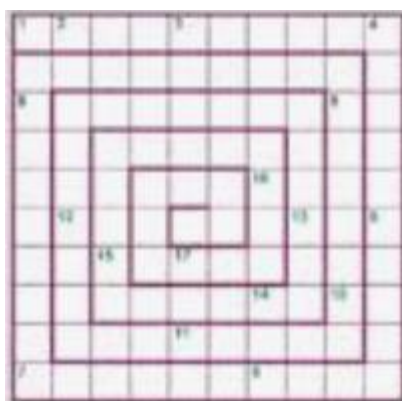


Рисунок 2 – Спиральная форма представления линейного кроссворда

2. Обзор существующих *систем-аналогов* – неотъемлемая часть описания предметной области, т.к. позволяет разработчику определить основные концепции, которые необходимо будет реализовать в системе. Описание должно приводиться с указанием отличительных особенностей разработанных систем, с перечислением их достоинств и недостатков, в отчете обязательно приводятся экранные формы этих систем.

² Рекомендуется рассказать о 4-5 разновидностях наиболее популярных кроссвордов.

3. Результатом последнего этапа является *диаграмма объектов предметной области* и краткое описание их свойств и функций. При построении данной диаграммы³ нужно помнить о том, что в данном случае объект – это «конкретная материализация абстракции», а не экземпляр класса, т.е. пока это понятие не программистское. Диаграмма объектов представляет статическую составляющую взаимодействующих между собой объектов, она должна включить в себя только те объекты предметной области, которые потом преобразуются в диаграмму классов. Связи между объектами показывают отношения между ними, при необходимости в диаграмме можно привести и атрибуты (свойства) объектов. На рисунке 3 приведен фрагмент диаграммы объектов для рассматриваемой в качестве примера системы.

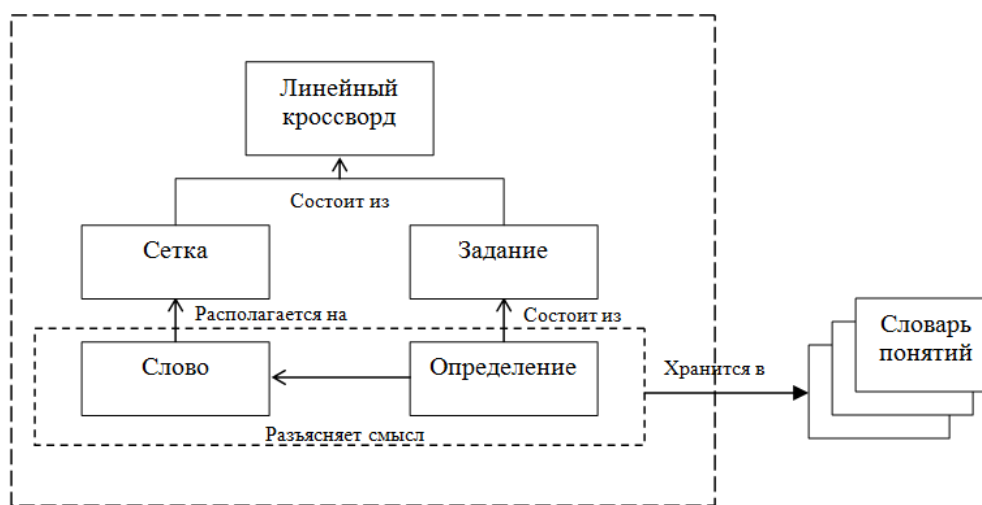


Рисунок 3 – Фрагмент диаграммы объектов предметной области

Коротким пунктиром выделены те сущности, которые могут представлять собой единое целое, длинным – внутренние объекты системы.

Диаграммы объектов не позволяют полностью описать объектную структуру системы, поэтому при их использовании нужно сосредоточиться на изображении интересующих вас наборов конкретных объектов [9].

Комментарии к примеру. Как видно из диаграммы, ЛК представляет собой (как и любой кроссворд) совокупность двух составляющих: *сетки*, на которой будут располагаться *слова*, и *задания*, которое по существу представляет собой набор *определений*, которые разъясняют смысл слов (понятий). Набор понятий и определений хранятся во внешнем файле (чаще всего текстовом), который мы ранее называли словарем понятий (см. лабораторную работу №1).

³ Диаграмма - это графическое представление множества элементов.

ЛАБОРАТОРНАЯ РАБОТА № 3

ПОСТАНОВКА ЗАДАЧИ

Постановка задачи – заключительный этап первой фазы ЖЦ системы. На данном этапе формулируются все требования, которым должна удовлетворять система. Постановка задачи пишется в повествовательной форме *в будущем времени* на основе ТЗ, в ней должны быть обязательно взаимоувязаны виды автоматизируемой деятельности (с привязкой к объекту(ам) автоматизации) со всеми ограничениями, накладываемыми на них, учтены особенности разрабатываемого информационного обеспечения и перечислены функции, которые должна выполнять система (с привязкой к процессам и информационному обеспечению).

Комментарии к примеру. Сначала необходимо сформулировать саму задачу, которая стоит перед разработчиками, обычно она включает в себя название проекта: «Перед авторами поставлена задача – разработать автоматизированную систему составления и разгадывания линейного кроссворда по выбранной теме, которая позволит ...». Далее последовательно описывается все процессы (виды автоматизируемой деятельности), в той последовательности, которая позволит получить требуемый результат. Также должно быть указано, какой тип системы будет реализован, например, «Система будет реализована в виде веб-приложения».

Если в системе предусмотрено разделений ролей, то необходимо об этом написать, например:

«В системе должно быть реализовано две роли пользователей: администратор и игрок. Каждый пользователь должен авторизоваться в системе, введя логин (от 4 до 8 символов) и пароль (от 4 до 10 символов). Игрок должен сначала зарегистрироваться в системе, введя логин, пароль и ... (указать дополнительные сведения, если он будет с ограничениями на них). Система должна проверить учётные данные пользователей и, в случае успеха, настроить интерфейс пользователя на заданную роль, в противном случае выдать сообщение об ошибке».

Далее необходимо описать, как будут реализованы процессы (для разрабатываемой системы таких процессов 3), каждый из них может функционировать независимо от других с привязкой к ролям.

Далее по тексту

«Режим администратора

Основные задачи администратора – создание кроссворда и работа со словарями понятий.

Составление кроссворда может проводиться в двух режимах: автоматическом (генерирование) и ручном. В любом случае пользователь должен предварительно задать параметры кроссворда⁴: определить его максимальную длину (не менее 15 и не более 255 символов), выбрать форму его представления (линейная, спиральная, ...), определить количество букв в пересечении (от 1 до 3), подключить словарь понятий (он хранится во внешнем текстовом файле определенной структуры⁵). При этом система должна провести проверку правильности этой структуры и, в случае несоответствия, выдать предупредительное сообщение (см. лабораторную работу №5, описание исключительных ситуаций) и обеспечить повторный ввод параметров. В системе должен осуществляться контроль типов и диапазонов значений параметров.

В режиме ручного составления кроссворда (или его редактирования) пользователю должны предоставляться следующие возможности: удаление слова (последнего), добавление нового слова (в конец), редактирование (замена) слова, если оно стоит в середине⁶. При этом должна быть обеспечена навигация по словам либо непосредственно на сетке кроссворда, либо с помощью специальных элементов управления⁷. В случае необходимости пользователь должен иметь возможность сохранить полученный кроссворд в файл (с целью дальнейшего его разгадывания или редактирования), структура файла должна быть определена в ходе проектирования⁵. Необходимо также предусмотреть контроль целостности создаваемого кроссворда (отсутствие пустых мест в середине кроссворда).

Работа со словарями понятий...

...

Режим игрока

Разгадывание кроссворда...»

Если в системе будет использоваться **база данных** (БД), то необходимо об этом написать, перечислить основные сущности БД и их атрибуты, а также перечислить справочники (номенклаторы), которые будут вестись в системе⁸.

Далее по тексту

⁴ см. раздел 2.1 ТЗ «Характеристики объекта автоматизации»

⁵ см. раздел 2.2 ТЗ «Требования к информационному обеспечению системы»

⁶ Последняя функция не обязательна, т.к. достаточно сложна при реализации.

⁷ см. раздел 2.5.1 ТЗ «Функции, реализуемые системой».

⁸ см. раздел 2.4 ТЗ «Требования к информационному обеспечению системы»

«Все данные, необходимые для функционирования системы, должны храниться в БД, ее структура определяется на основании следующих сведений:

- об игроке (логин, пароль, ...);
- о кроссворде (название, дата создания, ...);
-

В системе также должна быть обеспечена возможность получения справочной информации как о самой системе, так и предоставляемых ею возможностях⁹.

Таким образом, система должна выполнять следующие функции¹⁰: «

(здесь перечисляются все функции, которые были определены в разделе 5.1 ТЗ)».

⁹ Это должно быть у всех

¹⁰ Эта часть постановки задачи обязательна.

ЛАБОРАТОРНАЯ РАБОТА № 4

РАЗРАБОТКА СТРУКТУРЫ СИСТЕМЫ

Построение структурной схемы программной системы. На данном этапе система по функциональному признаку разделяется на основные подсистемы, между ними указываются информационные связи и/или связи по управлению, описывается основное назначение подсистем. При разработке структурной схемы используется методология структурного проектирования, в основе которой лежит алгоритмическая декомпозиция и иерархия вида «часть-целое», учитывающая, что внутренние связи элементов внутри подсистем сильнее, чем связь между подсистемами. Декомпозиция системы может повторяться многократно, вплоть до уровня конкретных процедур, при этом должна быть обеспечена целостность системы, а все составляющие компоненты взаимоувязаны. Для этого используются такие принципы разработки, как «сверху-вниз», «разделяй и властвуй», «иерархическое упорядочивание» и другие [3].

Дадим некоторые определения.

В первом приближении можно придерживаться нормативного понятия системы. **Система** (греч. - «составленное из частей», «соединение» от «соединяю») - множество элементов, находящихся в отношениях и связях друг с другом, которое образует определённую целостность, единство [11]¹¹.

Как следует из определения, отличительным (главным свойством) системы является ее *целостность*: комплекс объектов, рассматриваемых в качестве системы, должен обладать общими *свойствами* и *поведением*. Очевидно, необходимо рассматривать и связи системы с *внешней средой*. В самом общем случае понятие «система» характеризуется:

- наличием множества элементов;
- наличием связей между ними;
- целостным характером данного устройства или процесса.

Система должна представлять собой совокупность элементов (объектов, субъектов), находящихся между собой в определенной зависимости и составляющих некоторое единство (целостность), направленное на достижение определенной цели. Система может являться элементом другой системы более высокого порядка (*надсистема*) и включать в себя системы более низкого порядка (*подсистемы*). То есть систему можно рассматривать как набор подсистем, организованных для достижения определенной цели и описанных с помощью набора моделей (возможно, с различных точек зрения), а подсистему – как группу элементов,

¹¹ Большой Российский энциклопедический словарь, с. 1437.

часть которых составляет спецификацию поведения, представленного другими ее составляющими [9].

К типовым можно отнести следующие подсистемы:

- 1) подсистему управления;
- 2) подсистемы ввода-вывода:
 - a) подсистему настройки параметров;
 - b) файловую подсистему;
 - c) подсистему визуализации;
 - d) подсистему документирования;
 - e) подсистему взаимодействия с базой данных;
- 3) справочную подсистему.

Полученная в результате декомпозиции структура системы должна сопровождаться кратким описанием включенных в нее подсистем.

Комментарии к примеру. Приведем примерную структурную схему разрабатываемой системы и описание некоторых подсистем (рисунок 4).

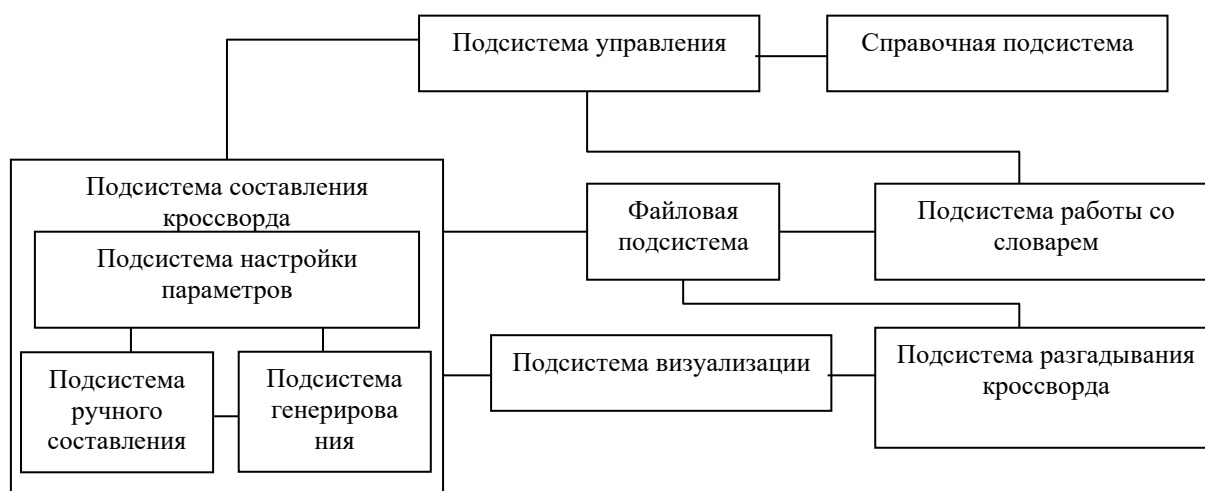


Рисунок 4 – Структурная схема системы (настольное приложение)

В состав системы входят следующие подсистемы:

- 1) подсистема управления, которая отвечает за взаимодействие подсистем между собой и представлена в виде иерархического меню;
- 2) подсистема составления кроссворда, в состав которой входят:
 - подсистема настройки параметров, которая отвечает за ввод (выбор) значений параметров кроссворда и проверку корректности этих значений;
 - подсистема ручного составления, которая ...
 - подсистема генерирования, которая ...

- 3) подсистема разгадывания, которая ...
- 4) файловая подсистема, которая ...
- 5) подсистема работы со словарем, которая ...
- 6) подсистема визуализации, которая ...
- 7) справочная подсистема, которая содержит сведения о системе (руководство пользователю) и ее об ее разработчиках.

ЛАБОРАТОРНАЯ РАБОТА № 5

РАЗРАБОТКА СПЕЦИФИКАЦИИ ТРЕБОВАНИЙ

Разработка спецификации программного обеспечения является одним из фундаментальных процессов технологии разработки ПО. Этот процесс анализа, формирования, документирования и проверки функциональных возможностей и ограничений системы называется в терминологии программной инженерии «разработка требований» (спецификация требований). Он является критическим этапом в создании всех видов программных систем, что обусловлено тем, что ошибки, допущенные на этой стадии, ведут к возникновению серьезных проблем на этапах проектирования и разработки. Опыт индустрии информационных технологий однозначно показывает, что вопросы, связанные с управлением требованиями, оказывают критически-важное влияние на программные проекты, в определенной степени - на сам факт возможности успешного завершения проектов. Только систематичная работа с требованиями позволяет корректным образом обеспечить моделирование задач реального мира и формулирование необходимых приемочных тестов для того, чтобы убедиться в соответствии создаваемых программных систем критериям, заданным реальными практическими потребностями.

Дадим некоторые определения.

Требования - это свойства, которыми должно обладать ПО для адекватного определения функций, условий и ограничений выполнения ПО, а также объемов данных, технического обеспечения и среды функционирования [12, 13].

На практике часто применяется подход, используемый в различных методологиях разработки ПО и базирующийся на определении групп требований к продукту. Такой подход обычно включает группы (типы, категории) требований, например: программные, системные, функциональные, нефункциональные (в частности, атрибуты качества) и т.п.

Программные требования (Software Requirements) - свойства программного обеспечения, которые должны быть надлежащим образом представлены в нём для решения конкретных практических задач [14]. Данная область знаний касается вопросов извлечения (сбора), анализа, специфицирования и утверждения требований к разрабатываемой ПС.

Функциональные требования задают «что» система должна делать; *нефункциональные* – с соблюдением «каких условий» (например, скорость отклика при выполнении заданной операции). При разработке этих требований в первую очередь необходимо учитывать потребности пользователя (заказчика). *Пользовательские требования* (User Requirements) – описывают цели/задачи пользователей системы, которые должны достигаться/выполняться

пользователями при помощи создаваемой программной системы. Часто пользовательские требования представляют в виде сценариев (вариантов использования) Use Case (см. лабораторную работу №5).

Среди нефункциональных требований на первый план выходят атрибуты качества и ограничения. *Атрибуты качества (Quality Attributes)* описывают дополнительные характеристики продукта в различных «измерениях», важных для пользователей и/или разработчиков. Атрибуты касаются вопросов портируемости, интероперабельности (прозрачности взаимодействия с другими системами), целостности, устойчивости и т.п. Данный вид требований мы будем называть *спецификацией качества*. *Ограничения (Constraints)* включают в себя формулировки условий, модифицирующих требования или наборы требований, сужая выбор возможных решений по их реализации. В частности, к ним могут относиться параметры производительности, влияющие на выбор платформы реализации и/или развертывания (протоколы, серверы приложений, баз данных, ...), которые, в свою очередь, могут относиться, например, к внешним интерфейсам.

Спецификация требований к ПО (SRS) - процесс формализованного описания функциональных и нефункциональных требований, требований к характеристикам качества в соответствии со стандартом качества ISO/IEC 9126-94, которые будут отрабатываться на этапах ЖЦ ПО.

В спецификации требований отражается:

- структура ПО;
- требования к функциям, качеству и документации;
- задается в общих чертах архитектура системы и ПО, алгоритмы, логика управления и структуры данных.

Характеристики правильно составленной спецификации требований к ПО

Спецификация требований должна быть [15]:

- 1) *корректной* (каждое требование, изложенное в ней, является требованием, которому должно удовлетворять программное обеспечение);
- 2) *однозначной* (каждое изложенное в ней требование может интерпретироваться только однозначно. Как минимум, для этого требуется, чтобы каждая характеристика конечного продукта была описана с использованием одного уникального термина);
- 3) *полной* (все существенные требования должны быть подтверждены и обработаны любые внешние требования, налагаемые спецификацией системы; определены отклики программного обеспечения на все классы входных данных, которые могут быть реализованы, во всех возможных ситуациях, как на допустимые, так и недопустимые

входные значения; полные обозначения и ссылки на все рисунки, таблицы и схемы в SRS и определение всех терминов и единиц измерения);

4) *непротиворечивой* (никакой набор отдельных требований, описанных в ней, не находится в противоречии с ней или с каким-то документом более высокого уровня);

5) *упорядоченной* по ее значимости и/или устойчивости (каждое требование должно иметь идентификатор и относиться к определенному классу требований: необходимые, условные и необязательные);

6) *проверяемой* (каждое требование, изложенное в ней, может быть проверено.);

7) *модифицируемой* (структура и стиль SRS таковы, что любые изменения требований могут быть выполнены легко, полностью и непротиворечивым образом при сохранении структуры и стиля);

8) *отслеживаемой* (должен четко прослеживаться источник каждого из ее требований и SRS облегчает обращение к каждому из требований при дальнейшей разработке или модернизации документации).

В процессе работы с требованиями участвуют так называемые «заинтересованные лица»¹², к числу которых мы будем относить:

- *пользователей* (людей, кто будет непосредственно использовать ПО; пользователи могут описать задачи, которые они решают (планируют решать) с использованием ПС, а также ожидания по отношению к атрибутам качества, отображаемые в пользовательских требованиях, в этой роли выступает преподаватель);

- *заказчиков* (людей, которые являются целевой аудиторией на рынке программного обеспечения, в этой роли выступает преподаватель);

- *инженеров по программному обеспечению* (людей, ответственных за техническую оценку путей решения поставленных задач и последующую реализацию требований заказчиков, в этой роли выступают студенты - члены команды разработчиков).

Все заинтересованные лица должны прийти к соглашению о том, какая система должна быть создана, чтобы в ней воплотились необходимые дидактические и технологические свойства. Данные соглашения фиксируются в документе, с которым могут сверяться и на который могут ссылаться все участники проекта, - интегрированная спецификация требова-

¹² Заинтересованное лицо - некто, имеющий возможность (в том числе, материальную) повлиять на реализацию проекта/продукта

ний, созданная на основе совместного использования традиционной функциональной спецификации и спецификации качества системы.

Комментарии к примеру. В спецификации качества перечисляются основные требования по показателям качества:

- описывается уровень надежности ПС;
- формулируются требования по быстродействию;
- требования к разработке интерфейса и т.п.

Функциональная спецификация включает в себя:

– перечень всех функций системы с привязкой их к конкретной подсистеме и к информационной среде (входные и выходные данные), фрагмент функциональной спецификации приведен в таблице 1;

– перечень исключительных ситуаций и реакцию системы на их возникновение, при необходимости приводится перечень ошибок, которые могут возникать в системе и соответствующие им системные сообщения, примеры исключительных ситуаций приведены в таблице 2.

Функциональная спецификация должна в полном объеме отображать информационные связи проектируемой системы как с внешним миром, так и между подсистемами. При необходимости расписываются информационные связи для сложных подсистем (спецификация второго уровня).

Исключительная ситуация – это ситуация, при которой система не может выполнить возложенных на нее функций или которая может привести к денормализации работы системы.

Таблица 2 – Перечень исключительных ситуаций

Название подсистемы	Название исключительной ситуации	Реакция системы
1 Справочная	1.1 Не возможно открыть файл справки	Выдача сообщения «Файл справки поврежден»
	1.2 Не возможно найти файл справки	Выдача сообщения «Отсутствует файл справки»
2 Файловая	2.1 Попытка открытия файла с несобственным форматом	Выдача сообщения «Файл поврежден или недопустимого формата»
	2.2 Файл с заданным именем не существует	Выдача аналогичного сообщения
...	...	...

**МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ЛАБОРАТОРНОМУ ПРАКТИКУМУ
ПО ДИСЦИПЛИНЕ «ПРОГРАММНАЯ ИНЖЕНЕРИЯ»**

Таблица 1 – Перечень функций, выполняемых системой (фрагмент)

Название подсистемы	Название функции	Информационная среда			
		Входные данные		Выходные данные	
		Назначение (наименование)	Тип, ограничения	Назначение (наименование)	Тип, ограничения
1 Справочная	1.1 Выдать сведения о разработчиках	Сведения о разработчиках системы (ФИО, номер группы)	Текст (МЕМО)	Визуальное отображение информации	-
	1.2 Выдать сведения о системе	Файл справки	Текстовый (*.HTML)		
				Код ошибки	целое
2 Настройки параметров	2.1 Задать количество букв в пересечении	Диапазон количества букв: минимальное максимальное	Целое 1 3	Текущее значение букв в пересечении	Целое
	2.2 Подключить словарь понятий	Имя файла	Строка, *.dict	Список понятий и их определений	Динамический массив строк
				Код ошибки	Целое
	2.3 Задать длину кроссворда	Диапазон длин: минимальное максимальное	Целое 15 250	Текущее значение длины	Целое
	...	...	...	...	...
3 Файловая	3.1 Загрузить файл с кроссвордом	Имя файла	Строка, *.kros	Кроссворд	Объект, структура определяется в ходе проектирования
				Код ошибки	Целое

ЛАБОРАТОРНАЯ РАБОТА № 6

РАЗРАБОТКА ПРОТОТИПА ИНТЕРФЕЙСА ПОЛЬЗОВАТЕЛЯ СИСТЕМЫ

Интерфейс пользователя является одним из важнейших элементов программы, это та часть программы, которая находится у всех на виду. Недочеты в пользовательском интерфейсе могут серьезно испортить впечатление даже о самых многофункциональных программах. Именно поэтому разработке и проектированию пользовательского интерфейса нужно уделять особое внимание.

Некоторые программисты склонны оставлять дизайн интерфейса пользователя на потом, считая, что реальное достоинство приложения - его программный код, который и требует большего внимания. Однако часто возникает недовольство пользователей из-за неудачно подобранных шрифтов, непонятного содержимого экрана и скорости его прорисовывания, поэтому работу над интерфейсом также нужно воспринимать серьезно.

Для чего нужна разработка пользовательского интерфейса? Как минимум, для этого есть две причины:

- чтобы пользователи работали более продуктивно, программа должна быть простой в использовании;
- хороший интерфейс может стать преимуществом против конкурентов, плохой - послужить причиной неудачи всего проекта.

Процесс создания интерфейса начинается с определения целей проекта, а также внутренних и внешних обстоятельств, которые вы должны принять во внимание. Для того чтобы правильно расставить приоритеты, необходимо учитывать:

- опыт работы пользователей с компьютером, типовые ситуации использования;
- какая информация необходима и когда, какие результаты должны быть получены;
- технологию разработки и платформа, на которой будут работать пользователи.

Начальная фаза разработки: концептуальный дизайн

В этой фазе разработки Вы должны решить, какой интерфейс лучше всего будет подходить для достижения ваших целей - текстовый, графический или мультимедиа. Затем необходимо выбрать структуру взаимодействия, которая обеспечивает разные степени гибкости для пользователей. Обычно, чем гибче структура, тем больше она требует от пользователя обучения, понимания, и времени на работу с окнами (открыть, закрыть, разместить и т.д.).

Для выполнения начальной фазы разработки необходимо погрузиться целиком в задачи пользователей и создать бумажный прототип *навигационной модели*. Навигационная модель показывает, как необходимо распределять функции или задачи между окнами вашей программы, она определяет, как пользователи смогут перемещаться как между различными задачами, так и внутри отдельной задачи. На рисунке 6 приведен пример навигационной модели для системы «Клавиатурный тренажер с функциями администратора».

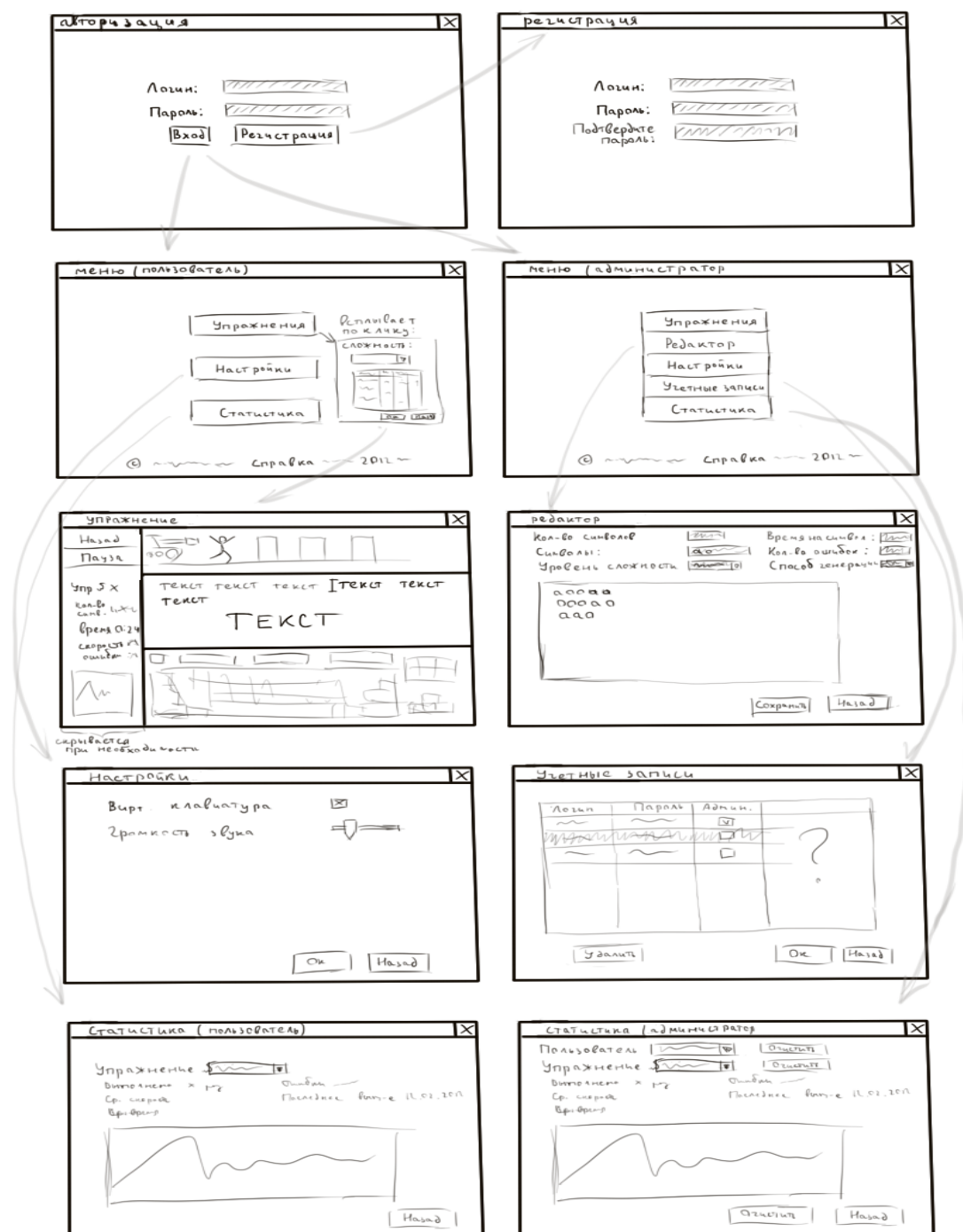


Рисунок 6 – Общее взаимодействие прототипов форм

Комментарии к примеру: Кроме навигационной модели в пояснительной записке должны быть кратко описаны все прототипы экранных форм.

Для того чтобы создать хороший интерфейс, на каждой стадии разработки необходима обратная связь от пользователей. Чтобы оценить концептуальную модель программы, необходимо показать ее схему пользователям и попросить объяснить ее вам. Если у пользователя возникнут трудности, значит, вы еще не достигли точки зрения пользователя в понимании проблемы.

Визуальный дизайн: использование компонентов

Хорошо выполненный дизайн выглядит чистым, простым и аккуратным. Его можно понять одним взглядом. Пользователь должен сразу распознавать, какие данные можно редактировать, какие нет; по каким объектам можно щелкать мышью и какие объекты можно перетаскивать.

Работа с несколькими формами

Если интерфейс пользователя должен содержать несколько форм, вам предстоит принять самое важное решение: какой использовать вид интерфейса - однок документный (SDI) или много документный (MDI).

В SDI-приложениях окна форм появляются совершенно независимо друг от друга (примером такого интерфейса может служить программа «Блокнот (Notepad)» или графический редактор MS Paint), в MDI-приложениях вы можете одновременно работать с несколькими объектами (примером такого интерфейса может служить текстовый процессор MS Word). Однако не имеет значения какой тип интерфейса SDI или MDI выбран; взаимодействие пользователя с формами происходит одинаково - посредством обработки событий, поступающих от элементов управления формы. Поэтому, если в вашем приложении предусмотрено несколько форм, программу необходимо написать так, чтобы у пользователей не было возможности нарушить предписанный ход ее выполнения (например, у пользователя не должно быть средств вывести форму, для которой еще не готова информация).

Эффективные меню

Еще одна важная часть разработки форм - создание содержательных и эффективных меню. Приведем некоторые важные рекомендации:

- следуйте стандартным соглашениям о расположении пунктов меню принятым в Windows File, Edit, View, и т.д.;
- группируйте пункты меню в логическом порядке и по содержанию;

- для группировки пунктов в раскрывающихся меню используйте разделительные линии;
- избегайте избыточных меню;
- избегайте пунктов меню верхнего уровня, не содержащих раскрывающихся меню;
- не забывайте использовать символ троеточия для обозначения пунктов меню, активизирующих диалоговые окна;
- обязательно используйте клавиатурные эквиваленты команд и «горячие» клавиши;
- помещайте на панель инструментов часто используемые команды меню.

О чем еще нужно помнить

Когда есть видимость работы приложения, пользователи более легко переносят длительное ожидание в работе программы. Один из способов информирования пользователя о ходе выполнения работы - использовать в форме *индикатор процесса*.

ЛАБОРАТОРНАЯ РАБОТА № 7

РАЗРАБОТКА ИНФОРМАЦИОННО-ЛОГИЧЕСКОГО ПРОЕКТА СИСТЕМЫ

Одной из широко используемых методик документирования требований является построение ряда моделей системы. Эти модели используют графические представления, показывающие решения как исходной задачи, так и разрабатываемой системы [16]. Как правило, графическое представление более понятно, чем описание требований на естественном языке.

Моделирование – это устоявшаяся и повсеместно принятая инженерная методика. Модели являются связующим звеном между процессом анализа и процессом проектирования системы. Что такое модель? *Модель* – это упрощенное представление реальности, по существу – это «чертеж» системы: в нее может входить как детальный план, так и более абстрактное представление системы «с высоты птичьего полета» [9]. Хорошая модель всегда включает элементы, которые существенно влияют на результат, и не включает те, которые малозначимы на данном уровне абстракции.

■ **Модель строится для того, чтобы лучше понимать разрабатываемую систему.**

Моделирование позволяет решить четыре различные задачи:

- 1) визуализировать систему в ее текущем или желательном для нас состоянии;
- 2) описать структуру или поведение системы;
- 3) получить шаблон, позволяющий сконструировать систему;
- 4) документировать принимаемые решения, используя полученные модели.

Моделирование предназначено не только для создания больших систем. От моделирования может выиграть любой проект. Даже при создании одноразовых программ, когда зачастую бывает полезно выбрать неподходящий код из-за преимущества в скорости разработки, которое дают языки визуального программирования, моделирование поможет коллективу разработчиков лучше представить план системы, а значит, выполнить проект быстрее и создать именно то, что подразумевал первоначальный замысел. Чем сложнее проект, тем более вероятно, что из-за отсутствия моделирования он свернется раньше времени – или будет создано не то, что нужно. Все полезные и интересные системы с течением времени обычно усложняются. Пренебрегая моделированием в самом начале создания системы можно серьезно пожалеть об этом, когда будет уже слишком поздно.

Моделирование имеет богатую историю во всех инженерных дисциплинах. Длительный опыт его использования позволил сформулировать *четыре основных принципа*:

1) выбор модели оказывает определяющее влияние на подход к решению проблемы и на то, как будет выглядеть это решение (различные точки зрения на мир приводят к созданию различных систем, со своими преимуществами и недостатками);

2) каждая модель может быть представлена с различной степенью точности (лучшей моделью будет та, которая позволяет выбрать уровень детализации в зависимости от того, кто и с какой целью на нее смотрит);

3) лучшие модели – те, что ближе к реальности (поскольку модель всегда упрощает реальность, задача в том, чтобы это упрощение не повлекло за собой какие-то существенные потери);

4) нельзя ограничиваться созданием только одной модели. Наилучший подход при разработке любой нетривиальной системы – использовать совокупность нескольких моделей, почти независимых друг от друга.

Унифицированный язык моделирования (Unified Modeling Language – UML) – это стандартный инструмент для разработки «чертежей» программного обеспечения. Его можно использовать для визуализации, спецификации, конструирования и документирования артефактов программных систем. UML подходит для моделирования любых систем – от информационных систем масштаба предприятия до распределенных Web-приложений и даже встроенных систем реального времени.

В лабораторном практикуме мы будем использовать UML для специфицирования (построения точных, недвусмысленных и полных моделей) системы и ее документирования. Язык UML предоставляет стандартный способ написания проектной документации на системы, включая концептуальные аспекты, такие как бизнес-процессы и функции системы, а также конкретные аспекты, такие как выражения языков программирования, схемы баз данных и повторно используемые компоненты ПО.

Язык UML не является языком программирования (он инвариантен к языкам программирования).

В нотации языка UML определены следующие виды канонических диаграмм:

- 1) вариантов использования (use case diagram);
- 2) классов (class diagram);
- 3) кооперации (collaboration diagram);
- 4) последовательности (sequence diagram);
- 5) состояний (statechart diagram);
- 6) деятельности (activity diagram);
- 7) компонентов (component diagram);

8) развертывания (deployment diagram).

Перечень этих диаграмм и их названия являются каноническими в том смысле, что представляют собой неотъемлемую часть графической нотации языка UML. Более того, процесс ООАП неразрывно связан с процессом построения этих диаграмм. При этом совокупность построенных таким образом диаграмм является самодостаточной в том смысле, что в них содержится вся информация, которая необходима для реализации проекта сложной системы. Каждая из этих диаграмм детализирует и конкретизирует различные представления о модели сложной системы в терминах языка UML.

Диаграмма вариантов использования представляет собой наиболее общую концептуальную модель сложной системы, которая является исходной для построения всех остальных диаграмм. Диаграмма классов, по своей сути, логическая модель, отражающая статические аспекты структурного построения сложной системы. Диаграммы кооперации и последовательностей представляют собой разновидности логической модели, которые отражают динамические аспекты функционирования сложной системы. Диаграммы состояний и деятельности предназначены для моделирования поведения системы. И, наконец, диаграммы компонентов и развертывания служат для представления физических компонентов сложной системы и поэтому относятся к ее физической модели.

Рассмотрим более подробно каждую из перечисленных диаграмм.

Диаграмма вариантов использования

Визуальное моделирование с использованием нотации UML можно представить как процесс поуровневого спуска от наиболее общей и абстрактной концептуальной модели исходной бизнес-системы к логической, а затем и к физической модели соответствующей ПС. Вначале строится модель в форме так называемой диаграммы вариантов использования (*use case diagram*), которая описывает функциональное назначение системы и является исходным концептуальным представлением в процессе ее проектирования и разработки. На ней изображаются отношения между актерами и вариантами использования. Создание диаграммы вариантов использования имеет следующие цели:

- определить общие границы и контекст моделируемой предметной области на начальных этапах проектирования системы;
- сформулировать общие требования к функциональному поведению проектируемой системы;
- разработать исходную концептуальную модель системы для ее последующей детализации в форме логических и физических моделей;

– подготовить исходную документацию для взаимодействия разработчиков системы с ее заказчиками и пользователями.

Актер (actor) - согласованное множество ролей, которые играют внешние сущности по отношению к вариантам использования при взаимодействии с ними (это может быть любой объект, субъект или система, взаимодействующая с моделируемой бизнес-системой извне, т.е. человек, техническое устройство, программа и т.п.).

Вариант использования - внешняя спецификация последовательности действий, которые система или другая сущность могут выполнять в процессе взаимодействия с актерами (он определяет набор действий, совершаемый системой при диалоге с актером).

Цель спецификации варианта использования заключается в том, чтобы зафиксировать некоторый аспект или фрагмент поведения проектируемой системы без указания особенностей реализации данной функциональности. В этом смысле каждый вариант использования соответствует *отдельному сервису*, который предоставляет моделируемая система по запросу актера, т. е. определяет один из способов применения системы. Сервис, который инициализируется по запросу актера, должен представлять собой законченную последовательность действий. Это означает, что после того как система закончит обработку запроса актера, она должна возвратиться в исходное состояние, в котором снова готова к выполнению следующих запросов.

Диаграмма вариантов использования содержит конечное множество вариантов использования, которые в целом должны определять все возможные стороны ожидаемого поведения системы. Для удобства множество вариантов использования может рассматриваться как отдельный пакет. Применение вариантов использования на всех этапах работы над проектом позволяет не только достичь требуемого уровня унификации обозначений для представления функциональности подсистем и системы в целом, но и является мощным средством последовательного уточнения требований к проектируемой системе на основе их итеративного обсуждения со всеми заинтересованными специалистами.

Комментарии к примеру. На рисунке 7 приведен фрагмент диаграммы вариантов использования для разрабатываемой системы (он соответствует функциональной спецификации, приведенной в таблице 1, и дополняет ее новыми функциями).

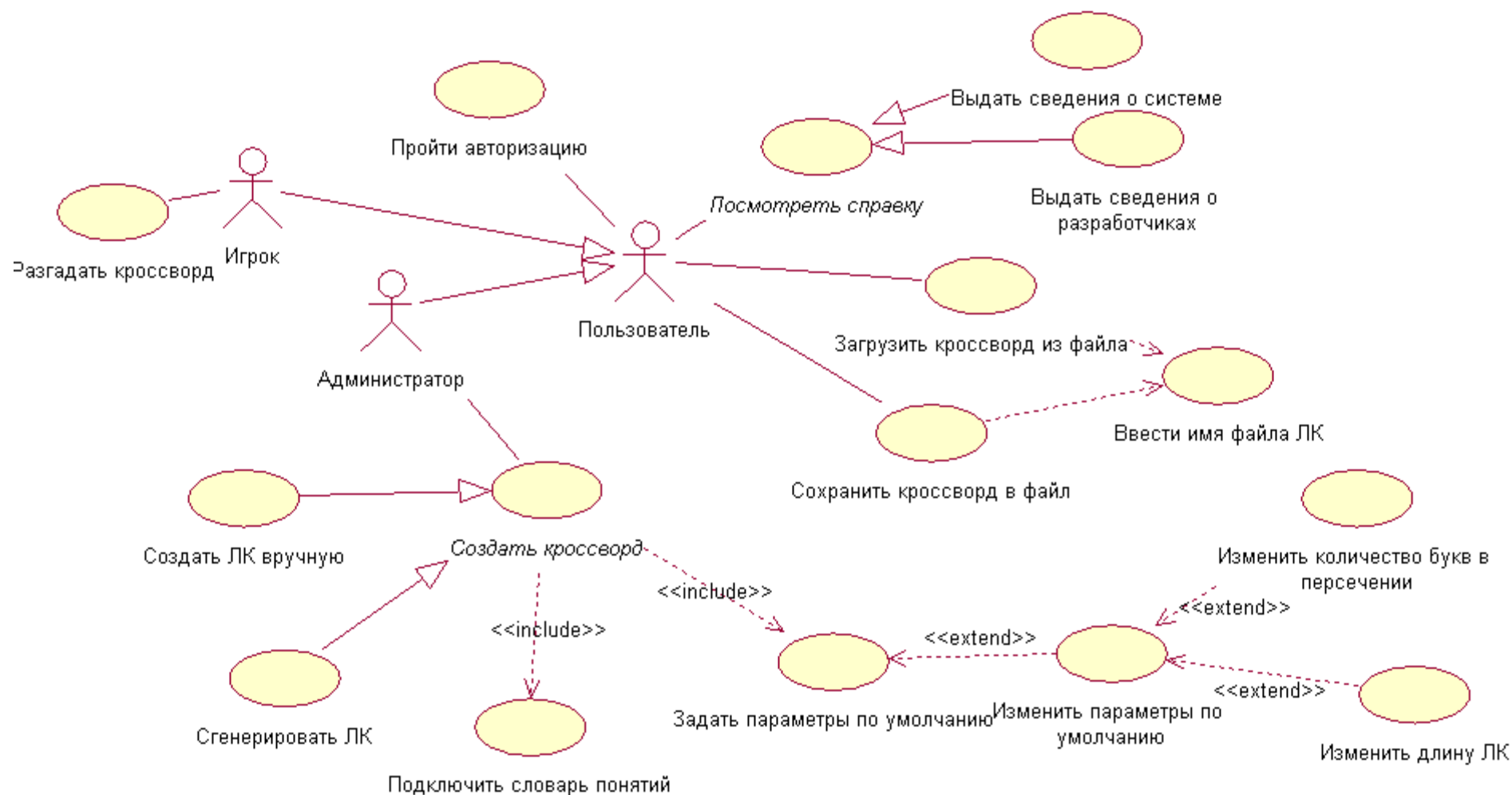


Рисунок 7 – Диаграмма вариантов использования системы (фрагмент)

Любая диаграмма, входящая в Ваш UML-проект должна быть прокомментирована: кратко описаны все основные сервисы, которые приведены на диаграмме. Например, «Перед тем как работать в системе пользователь должен пройти процедуру авторизации (ввести логин и пароль). Система проверит введенные данные и настроит интерфейс пользователя на соответствующую роль. В системе должны быть предусмотрены две роли пользователя: администратор и игрок. Любой пользователь системы должен иметь возможность посмотреть справочную информацию (сведения о разработчиках и сведения о самой системе). Администратор может создавать кроссворд в ручном или автоматическом режиме, для этого он должен подключить словарь понятий и определить параметры кроссворда (выбрать значения, заданные по умолчанию, или изменить параметры); и т.д.»

Следует отметить, что одним из требований языка UML является самодостаточность диаграмм для представления информации о моделях проектируемых систем. Однако большинство разработчиков и экспертов согласны с тем, что изобразительных средств языка UML явно не хватает для того, чтобы учесть на диаграммах вариантов использования особенности функционального поведения сложной системы. С этой целью рекомендуется дополнять этот тип диаграмм **текстовыми сценариями**, которые уточняют или детализируют последовательность действий, совершаемых системой при выполнении ее вариантов использования.

Сценарий (scenario) - определенная последовательность действий, которая описывает действия актеров и поведение моделируемой системы в форме обычного текста.

В контексте языка UML сценарий используется для дополнительной иллюстрации взаимодействия актеров и вариантов использования. Предлагаются различные способы представления или написания подобных сценариев. Рассмотрим один из шаблонов для применения на начальных этапах концептуального моделирования.

При написании сценариев вариантов использования важно понимать, что текст сценария должен дополнять или уточнять диаграмму вариантов использования, но не заменять ее полностью. В противном случае будут потеряны достоинства визуального представления моделей.

Сценарий состоит из следующих разделов:

1) главный раздел, в котором отражается имя варианта использования, перечисляются актеры, в нем участвующие, определяется цель и дается краткое описание. Кроме того, может быть указан тип сценария и даны ссылки на другие варианты использования.

2) раздел «Основной поток событий», в котором отражается типичный ход событий, приводящий к успешному выполнению варианта использования;

3) раздел «Альтернативы» («Исключения»);

4) раздел «Примечания».

Комментарии к примеру. Рассмотрим вариант использования «Пройти авторизацию» для автоматизированной системы составления линейного кроссворда (прототип экранной формы приведен на рисунке 8).

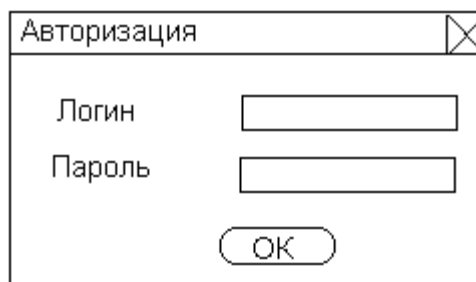


Рисунок 8 – Прототип экранной формы авторизации пользователя

Вариант использования: Пройти авторизацию

Краткое описание. Даёт возможность любому пользователю войти в систему по своему имени и паролю с настройкой интерфейса системы на соответствующие права доступа. Пользователь двойным щелчком левой кнопки мыши по пиктограмме «Кроссворд» запускает приложение, перед ним открывается окно авторизации.

Актант. Пользователь.

Предусловия. Компьютер пользователя включён, на экране – главное окно операционной системы с набором пиктограмм на рабочем столе.

Основной поток событий.

1) На экране появляется форма ввода имени пользователя и пароля с полями ввода «Логин», «Пароль» и с кнопками ОК и «х» (Выход).

2) Пользователь щелкает левой кнопкой мыши (ЛКМ) в поле «Логин».

3) Система делает данное поле активным и выводит красным цветом под ним информацию о допустимых символах и длине логина.

4) Пользователь вводит логин.

A1: Пользователь щелкает ЛКМ по кнопке «х».

A2: Пользователь щелкает ЛКМ по кнопке «ОК».

5) Пользователь щелкает левой кнопкой мыши (ЛКМ) в поле «Пароль».

6) Система делает данное поле активным и выводит красным цветом под ним информацию о допустимых символах и длине пароля.

7) Пользователь вводит пароль.

A1: Пользователь щелкает ЛКМ по кнопке «х».

A2: Пользователь щелкает ЛКМ по кнопке «ОК».

8) Пользователь щелкает ЛКМ по кнопке «ОК».

9) Система проверяет логин и пароль.

10) Система закрывает форму ввода имени и пароля и выводит на экран главную форму приложения с пунктами меню. Состав пунктов меню настраивается в соответствии с правами пользователя. Вариант использования завершается успешно.

A3: Имя и/или пароль неверны.

Альтернативы.

A1: Пользователь щелкает ЛКМ по кнопке «х».

A1.1. Система закрывает форму ввода имени и пароля и выводит на экран главное окно операционной системы. Вариант использования завершается.

A2: Пользователь щелкает ЛКМ по кнопке «ОК».

A2.1. Система выводит под каждым полем сообщение «Не введен логин/пароль».

A2.2. Переход к п.2 основного потока.

A3: Имя и/или пароль неверны.

A2.1. Система выводит сообщение о неверном вводе имени и/или пароля под соответствующим полем и просит повторить вход или выйти из приложения. В последнем случае вариант использования завершается.

Постусловия. При успешном завершении на экране – главная форма приложения с меню, настроенном на права пользователя.

Неясные вопросы. Уточнить права пользователей и настройки.

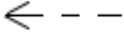
Диаграмма классов

Диаграммы классов – это наиболее часто используемый тип диаграмм, которые создаются при моделировании объектно-ориентированных систем, они показывают набор классов, интерфейсов и коопераций, а также их связи. На практике диаграммы классов применяют для моделирования статического представления системы, они служат основой для целой группы взаимосвязанных диаграмм – диаграмм компонентов и диаграмм размещения.

На данном этапе для всех информационных объектов, выделенных в системе (см. п.1), разрабатываются классы с указанием полей, методов и свойств, которые регулируют процессы обработки данных (потoki данных заданной структуры) и/или структуры данных.

На рисунке 9 приведен пример диаграммы классов на этапе проектирования. На ней определены основные сущности системы с указанием отношений между ними. В методологии UML приняты следующие обозначения для отношений между классами (см. таблицу 3).

Таблица 3 – Основные виды отношений между классами

Название отношения	Обозначение
Зависимости (dependency)	
Обобщения (generalization)	
Ассоциации (association)	
Агрегации (association)	
Композиции (composition)	

Отношение зависимости используется тогда, когда изменение одного элемента модели может потребовать изменения другого, зависящего от него. Это наиболее общая форма отношений, все другие виды отношений можно считать частным случаем данного.

Отношение обобщения показывает, что некоторые объекты являются потомками базового (родительского) класса.

Отношение ассоциации показывает, что некоторые объекты образуют группу (ассоциацию). Наиболее простая ассоциация - бинарная. Для данного вида отношения может быть указана мощность связи (например, 1..*, один ко многим).

Отношения агрегации и композиции рассматриваются как частный случай ассоциации. *Агрегация* – вид отношения, при котором один класс включает в себя в качестве составляющей другие классы (при этом используется вид декомпозиции «часть-целое»). Можно дать такое определение отношения агрегации [9]: «Агрегация – это отношение «часть-целое» между двумя равноправными объектами, когда один объект (контейнер) имеет ссылку на другой объект. Оба объекта могут существовать независимо: если контейнер будет уничтожен, то его содержимое – нет». При этом связь между объектами устанавливается на уровне ссылок. *Композиция* – частный случай агрегации, отличие заключается в том, что этом включаемый объект может существовать только как часть контейнера (целого), т.е. связь между объектами организуется «по значению».

**МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ЛАБОРАТОРНОМУ ПРАКТИКУМУ
ПО ДИСЦИПЛИНЕ «ПРОГРАММНАЯ ИНЖЕНЕРИЯ»**

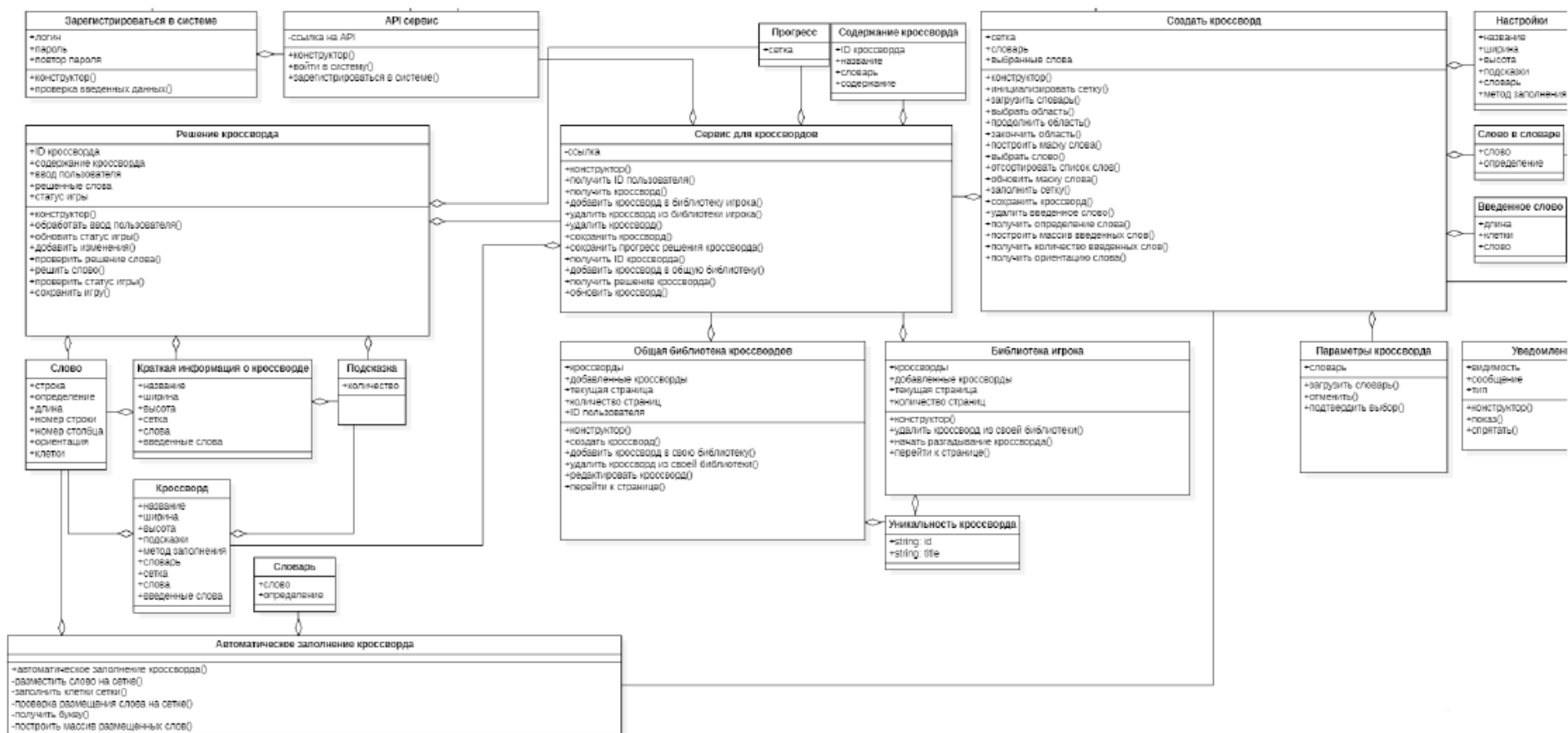


Рисунок 9 – Пример фрагмента диаграммы классов системы (спецификация)

Для того чтобы понять назначение данных сущностей, входящих в диаграмму классов, необходимо представить в табличном виде описание всех классов (см. таблицу 4).

Таблица 4 – Описание классов системы (пример)

Название класса	Назначение
Кроссворд	Класс предназначен для хранения информации о кроссворде
Словарь	Класс предназначен для хранения информации о словаре
...	...

Существуют различные разновидности классов:

- *абстрактный (abstract) класс* не имеет экземпляров или объектов, для обозначения его имени на диаграмме используется наклонный шрифт (курсив);
- *активный класс (active class)* – класс, каждый экземпляр которого имеет свою собственную нить управления;
- *пассивный класс (passive class)* – класс, каждый экземпляр которого выполняется в контексте некоторого другого объекта;
- *квалифицированное имя (qualified name)* используется для того, чтобы явно указать, к какому пакету относится тот или иной класс. Для этого применяется специальный символ в качестве разделителя имени – двойное двоеточие «::»;
- *имя класса* без символа разделителя называется простым именем класса.

На рисунке 10 приведен пример диаграммы классов на этапе реализации.

**МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ЛАБОРАТОРНОМУ ПРАКТИКУМУ
ПО ДИСЦИПЛИНЕ «ПРОГРАММНАЯ ИНЖЕНЕРИЯ»**

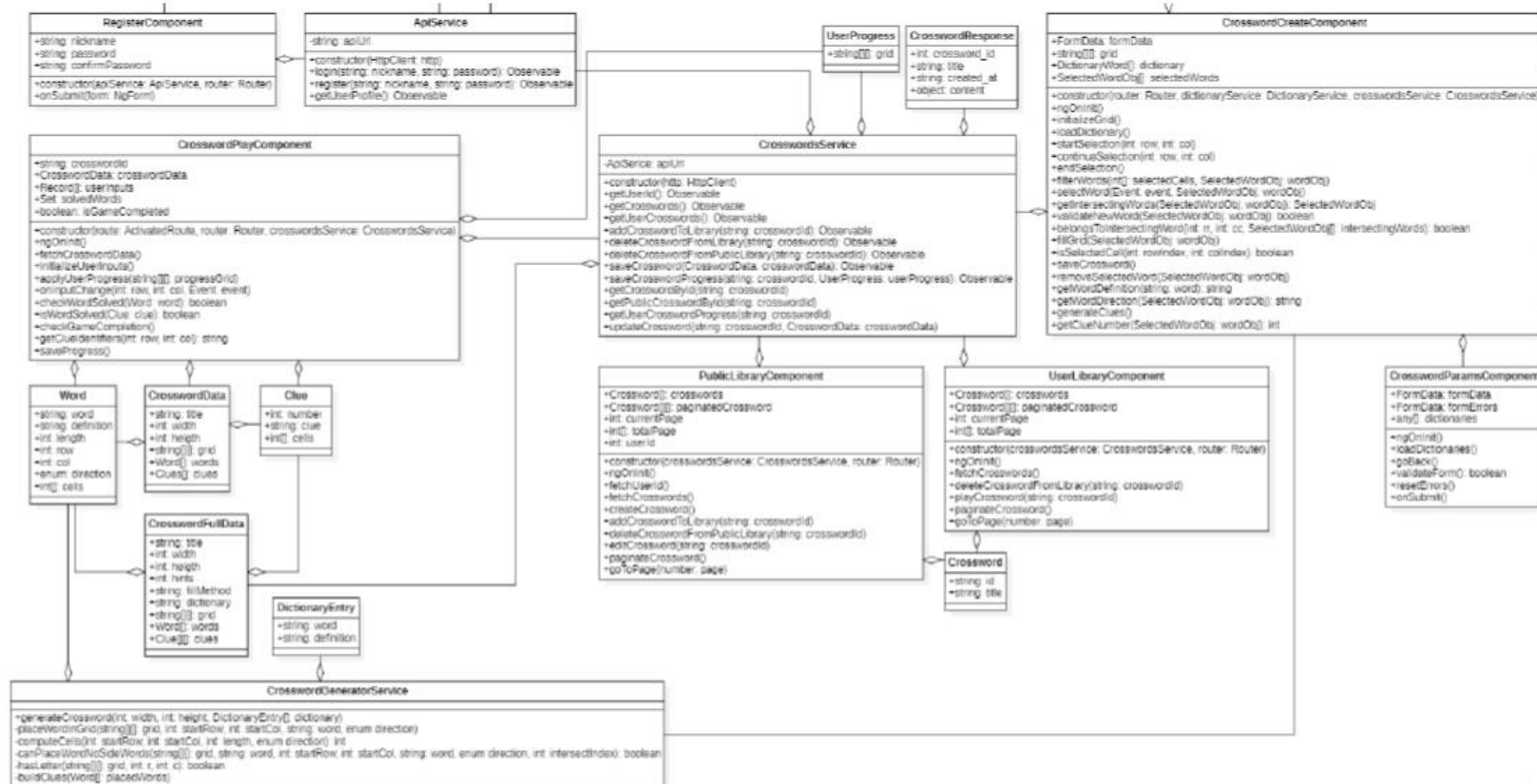


Рисунок 10 – Пример фрагмента диаграммы классов системы (реализация)

ЛАБОРАТОРНАЯ РАБОТА № 8

РАЗРАБОТКА АЛГОРИТМОВ ОБРАБОТКИ ДАННЫХ

Выбор и обоснование алгоритмов (или разработка и описание алгоритмов). Если для организации работы системы можно использовать уже известные алгоритмы, то необходимо провести их сравнительный анализ (по эффективности) и выбрать наилучший для данной системы (при введенных ограничениях). В противном случае пользователь разрабатывает свои алгоритмы, обосновывая их необходимость. Описание алгоритма ведется в вербальной форме и с помощью схем алгоритмов [7].

Описание логической модели данных. Если в проекте данные необходимо хранить в базе данных (БД), то на данном этапе должна быть разработана концептуальная и логическая модель БД, выделены и описаны основные сущности, определены между ними отношения. Модели должны быть представлены в соответствующей нотации (ER-модель (сущность - связь), SHM-модель (семантическую иерархическую модель) [3]). Переход к реляционной модели производится в соответствии с правилами, приведенными в [4]. Обязательным условием является нормализация реляционной модели информационной базы системы.

Физическое проектирование программной системы - завершающий этап разработки системы. Он включает в себя:

- разработку пользовательского меню, которое должно быть ориентировано на структуру системы;
- описание интерфейса с обоснованием выбора того или иного стандарта оформления /1/.
- разработка модулей системы и описание их спецификаций, взаимодействие модулей должно быть представлено в виде диаграммы модулей с указанием иерархии модулей.

Реализация проекта и предъявление ПС (подсистемы) руководителю. Реализация проекта производится строго в соответствии с логическим проектом по технологии быстрой разработки приложений RAD (Rapid Application Development), в основе которой лежит спиральная модель жизненного цикла ПС, в определенной среде разработки, при необходимости используются дополнительные инструментальные средства (например, CASE-инструменты в виде специализированных пакетов и сред проектирования), производится автономная и комплексная отладка и тестирование. Руководитель проверяет полноту и качество реализации функций, соответствие системы техническому заданию и логическому проекту. Для демонстрации работоспособности системы необходимо подготовить нескольких тестовых примеров. При необходимости производится доработка реализации с повторным предъявлением системы, после доработки система выносится на защиту.

Полное оформление документации проекта и защита проекта. После приемки реализации студент оформляет пояснительную записку к ПС со всеми требуемыми приложениями.

ОФОРМЛЕНИЕ ОТЧЕТА

Пояснительная записка к проекту оформляется в соответствии со стандартом университета [21] и должна содержать:

- титульный лист (пример оформления титульного листа приведен в приложении В);
- задание на разработку программной системы (пример технического задания приведен в приложении А);
- реферат (пример реферата приведен в приложении Г);
- содержание отчета (структура содержания приведена в приложении Б);
- введение;
- основная часть;
- заключение;
- перечень принятых сокращений (при наличии);
- перечень принятых терминов (при наличии);
- список использованных источников;
- приложения.

Основная часть пояснительной записки делится на разделы:

- 1) описание и анализ предметной области;
- 2) проектирование системы;
- 3) реализация системы.

Во введении раскрывается назначение системы, кратко описываются ее характеристики, актуальность разработки системы, а также приводятся существующие на текущий момент времени аналоги.

I. В разделе «Описание и анализ предметной области» отражаются результаты выполнения лабораторной работы №1: даются базовые понятия и определения, описание применяемых методов и математических моделей (при необходимости), описываются системы-аналоги, выделяются объекты системы и их взаимосвязи между ними.

III. В разделе «Проектирование системы» отражаются результаты

- 1.1 разработка структурной схемы системы, в которой описывается назначение всех подсистем;
- 1.2 функциональная спецификация ПС уточняет структурную схему системы, в ее состав входит перечень функций, выполняемых системой; описание внешней информационной среды и перечень исключительных ситуаций (при необходимости);

1.3 разработка схемы функционирования ПС (с необходимой детализацией внутри подсистем);

1.4 разрабатываются структуры данных и классы объектов, их отношения представляются в виде диаграммы (иерархии) классов, при необходимости разрабатывается концептуальная и логическая модели хранения данных (ER-модель или SHM-диаграмма хранимых данных), определяются структуры потоков данных. Описываются все проектные решения по оптимизации выбранной модели хранения данных, а также по разработке логики процессов обработки данных и управления.

1.5 Производится выбор и обоснование (разработка и описание) алгоритмов, применяемых для обработки данных, описание алгоритмов выполняется с помощью граф-схем;

1.6 Производится выбор комплекса технических и обоснование архитектуры системы, сопровождаемое ресурсными расчетами (требуемый объем оперативной и внешней памяти) и расчетами быстродействия системы. Кроме того, выбираются системные и программных средства (языки программирования, среды разработки, системы управления базами данных, операционные системы). Если в задании оговариваются дополнительные требования к системе по точности, надежности и другим показателям, то в записке должны присутствовать соответствующие расчеты и обоснования, показывающие, что проектируемая система удовлетворяет требованиям задания.

II. **В разделе «Реализация системы»** обосновываются решения, принятые при реализации логического проекта системы:

2.1 Производится описание структуры пользовательского меню, входных и выходных форм интерфейсной части системы.

2.2 Приводится *реализация* всех структур данных и классов, используемых в системе. Если в системе использовалась БД, то должно быть приведено описание физической модели данных с указанием объемов памяти, необходимых для хранения таблиц, приводятся описание основных запросов, подтверждающих правильность концептуальной модели данных.

2.3 Производится разработка структуры программы на модульном (компонентном) уровне, описываются способы взаимодействия и особенности реализации компонентов, приводится диаграмма компонентов и их описание (в частности указывается, в состав какой подсистемы входит каждый компонент/модуль).

2.4 Разрабатывается тестовый пример и приводятся результаты тестирования системы с наглядным отображением результатов тестирования в виде таблиц, диаграмм, экранов с пояснительным текстом. Разрабатываются и описываются в соответствии со стандартами [21, 22].

Термины и определения должны соответствовать ГОСТ 34.003-90 [2].

Разделы основной части для удобства чтения разбиваются на подразделы с заголовками, соответствующими содержанию подраздела. Подразделы могут быть далее разбиты на отдельные пункты.

В *приложения* выносятся:

- листинги программ;
- руководство по эксплуатации системы;
- текст контрольного примера и результаты тестирования системы;
- другие материалы, размещение которых в основной части затрудняет чтение пояснительной записки.

Рекомендуемый объем пояснительной записки 70-80 страниц машинописного текста (без приложений).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Вендров, А. М. CASE-технологии. Современные методы и средств проектирования информационных систем. [Текст] / А. М. Вендров – М.: Финансы и статистика, 1999. – 256 с.
- 2 Информационная технология. Комплекс стандартов и руководящих документов на автоматизированные системы [Сборник]: //Сборник ГОСТ 34.003-90, РД 50-680-88, РД 50-682-89, ГОСТ 34.201-89 - ГОСТ 34.602.89. – М.: Изд-во стандартов, 1992. –150 с.
- 3 Зеленко, Л.С. Технологии программирования и программная инженерия (1 часть) [Текст]: учебное пособие / Л.С. Зеленко. – Самара: изд-во СГАУ, 2006. – 96 с.: ил.
- 4 Леоненков, А. В. Нотация и семантика языка UML [Электронный ресурс]/ А.В. Леоненков. – Интернет-университет информационных технологий. <http://www.intuit.ru/department/pl/umlbasics>.
- 5 Определение линейного кроссворда [Электронный ресурс] - http://ru.wikipedia.org/wiki/Линейный_кроссворд.
- 6 СанПиН 2.2.2/2.4.2198-07. Гигиенические требования к персональным электронно-вычислительным машинам и организации работ. Изменение N 1 к СанПиН 2.2.2/2.4.1340-03 [Текст] – Дата введения с 01.07.07. – М.: Бюллетень нормативных и методических документов Госсанэпиднадзора. - № 3. – 2007.
- 7 ГОСТ 12.1.005-88. Система стандартов безопасности труда. Общие санитарно-гигиенические требования к воздуху рабочей зоны [Текст] – Дата введения 01.01.1989. – М.: Стандартиформ, 2006. – 49 с.
- 8 ГОСТ 12.1.007-76. Система стандартов безопасности труда. Вредные вещества. Классификация и общие требования безопасности [Текст] – Дата введения 01.01.1977. - М.: Стандартиформ, 2007. – 7 с.
- 9 Буч, Г. Язык UML. Руководство пользователя [Текст] /Г. Буч, Д. Рамбо, А. Якобсон. - 2-е изд.: Пер. с англ. Мухина Н. – М.: ДМК Пресс, 2006. – 496 с.: ил.
- 10 Определение кроссворда [Электронный ресурс] - <http://ru.wikipedia.org/wiki/Кроссворд>.
- 11 Большой Российский энциклопедический словарь [Текст]. - М.: БРЭ, 2003.
- 12 Вигерс, К. Разработка требований к программному обеспечению [Текст]: Пер. с англ. /К. Вигерс. – М.: Издательский торговый дом «Русская Редакция», 2004. 576с.: ил.
- 13 Зеленко, Л.С. Программная инженерия. Курс лекций [Текст]: учебное пособие / Л.С. Зеленко. – Самара: изд-во СГАУ, 2012. – 148 с.: ил.

14 Орлик, С. Основы программной инженерии (по SWEBOK). Программные требования [Электронный ресурс] - http://swbok.sorlik.ru/1_software_requirements.html.

15 Методика составления спецификаций требований к программному обеспечению, рекомендуемая Институтом Инженеров по Электротехнике и Радиоэлектронике (IEEE-830-1998) [Электронный ресурс] - <http://www.webisgroup.ru/services/programming/srs/ieee-830-1998/>.

16 Sommerwyl, I. Инженерия программного обеспечения/ Иан Соммервиль. – М., СПб, Киев: Издательский дом «Вильямс», 2002. – 626 с. : ил.

17 Смит, Дж. Принципы концептуального проектирования баз данных [Текст] // Дж. Смит, Д.Смит // Требования и спецификации в разработке программ. – М.: Мир, 1984. – С.165 - 198.

18 Джексон, Г. Проектирование реляционных баз данных для использования с микроЭВМ [Текст]: /Пер.с англ. – М.: Мир, 1991. – 252 с.

19 Леоненков, А.В. Самоучитель UML [Текст]. – СПб: БВХ-Петербург, 2002. –234 с.

20 Липаев, В.В. Программная инженерия. Методологические основы [Текст]. – М.: Издательство «ТЕИС», 2006. – 609 с.

21 СТО СГАУ 02068410-004-2007. Общие требования к учебным текстовым документам [Текст]: методические указания. – Самара: Изд-во Самар. гос. аэрокосм. ун-та, 2007. – 30 с.

22 ГОСТ 19.701-90 (ИСО 5807-85). ЕСПД. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. – М.: Изд-во стандартов, 1991. – 26 с.

ПРИЛОЖЕНИЕ А

Структура пояснительной записки

СОДЕРЖАНИЕ

Введение

1 Описание и анализ предметной области

1.1 Описание предметной области

1.2 Описание систем-аналогов

1.3 Диаграмма объектов предметной области

1.4 Постановка задачи

2 Проектирование системы

2.1 Выбор и обоснование архитектуры системы

2.2 Структурная схема системы

2.3 Спецификация системы

2.3.1 Функциональная спецификация

2.3.2 Перечень исключительных ситуаций

2.3 Разработка прототипа интерфейса пользователя

2.4 Информационно-логический проект системы

2.4.1 Язык UML

2.4.2 Диаграмма вариантов использования

2.4.3 Сценарии

2.4.4 Диаграмма состояний

2.4.5 Диаграмма деятельности

2.4.6 Диаграмма последовательности

2.4.7 Диаграмма классов

2.4.8 Логическая модель данных (при необходимости)

2.5 Выбор и обоснование комплекса программных средств

2.5.1 Выбор языка программирования и среды разработки

2.5.2 Выбор операционной системы

2.5.3 Выбор среды программирования

2.5.4 Выбор системы управления базами данных (при необходимости)

3 Реализация системы

3.1 Разработка и описание интерфейса пользователя

3.2 Реализация классов и структур данных

3.3 Диаграммы реализации

3.3.1 Диаграмма компонентов

3.3.2 Диаграмма развертывания

3.3.2 Диаграмма классов

3.4 Физическая модель данных (при необходимости)

3.4 Выбор и обоснование комплекса технических средств

3.4.1 Расчет объема занимаемой памяти

3.4.2 Минимальные требования, предъявляемые к системе

Заключение

Список использованной литературы

Приложение А Руководство пользователя

Приложение Б Листинг программы

ПРИЛОЖЕНИЕ Б

Пример оформления реферата

РЕФЕРАТ

Пояснительная записка 75 с, 14 рисунков, 5 таблиц, 6 источников, 2 приложения.

Графическая документация: 15 слайдов презентации.

ДЕРЕВО ПОИСКА, ГЕНЕРАТОР КРОССВОРДОВ, ГОЛОВОЛОМКА, СЛОВАРЬ
ТЕРМИНОВ, ВАРИАНТ ОТОБРАЖЕНИЯ, РАЗГАДЫВАНИЕ.

Объектом автоматизации является линейный кроссворд.

Во время курсового проектирования разработан информационно-логический проект и соответствующая им система, позволяющая выполнять ручное построение линейного кроссворда по заданной теме или его автоматическую генерацию, учитывая ограничения на параметры, заданные пользователем. Тема кроссворда выбирается пользователем в соответствии с содержанием словаря заданий. Задания (понятие и его расшифровка) хранятся во внешнем текстовом файле и могут дополняться вручную внутри программы, при этом ограничений на длину словаря не существует. В системе имеется возможность сохранения кроссвордов в файл с целью последующего их разгадывания.

В системе реализовано две роли пользователей: администратор и игрок. Администратор отвечает за создание кроссворда и работу со словарями понятий, игрок может разгадывать кроссворды.

Программа написана на языке C# в среде Visual Studio 2019 и функционирует под управлением операционной системы Windows 10 и выше. Доступ к данным осуществляется с помощью СУБД PostgreSQL 10.

Учебное издание

ПРИНЦИПЫ РАЗРАБОТКИ УЧЕБНЫХ ПРОГРАММ

Методические указания к лабораторному практикуму
по дисциплине «Программная инженерия»

Методические указания

Составитель: З е л е н к о Лариса Сергеевна

Самарский государственный аэрокосмический
университет им. академика С.П.Королева.

443086 Самара, Московское шоссе, 34.